INFORME ACADÉMICO – SEMANA 07

Asignatura: Arquitectura de Software / Base de Datos

Proyecto: Sistema de Atención Ciudadana (SAC)

Módulo / Dominio: `pe.gob.sac`

Tema: Diseño Físico de la Base de Datos NoSQL y Despliegue en Plataforma Cloud (Microsoft Azure Cosmos DB)

Estudiante: Percy Andy Hizarra Samaniego

Fecha: Octubre de 2026

1. INTRODUCCIÓN Y OBJETIVOS

1.1 Introducción

En el desarrollo de sistemas web orientados a la gestión de trámites municipales, la persistencia de datos debe ser escalable, de alta disponibilidad y segura. En el marco de la **Semana 07**, se aborda el **diseño físico de la base de datos NoSQL** para el *Sistema de Atención Ciudadana (SAC)*.

A partir de la arquitectura lógica basada en el paradigma de documentos con **MongoDB** (gestionada desde la capa de persistencia mediante la clase `MongoUtil`), se procede a la especificación física de las colecciones, estrategias de indexación, reglas de validación de esquemas (JSON Schema Validation) y el aprovisionamiento de la infraestructura en el proveedor de nube **Microsoft Azure** a través de **Azure Cosmos DB for MongoDB**.

1.2 Objetivos

- **General:** Diseñar e implementar el modelo físico de la base de datos orientada a documentos para el proyecto SAC y su despliegue en un entorno Cloud Enterprise (Microsoft Azure).
- **Específicos:**
    1. Estructurar el esquema físico BSON/JSON de las colecciones principales: `usuarios`, `solicitudes` y `encuestas`.
    2. Configurar los índices de búsqueda para optimizar el rendimiento de las operaciones I/O en lectura y escritura.
    3. Desplegar la instancia en la nube utilizando **Azure Cosmos DB (API para MongoDB)**.
    4. Documentar la integración técnica y almacenar los scripts de infraestructura en el repositorio de código fuente del proyecto.

2. ARQUITECTURA FÍSICA DE LA BASE DE DATOS (NoSQL - MONGODB)

La base de datos se denomina **`sac_db`**. Al ser una base de datos orientada a documentos, la estrategia de diseño físico opta por la **embebición (embedding)** para relaciones de alta coherencia y corta cardinalidad (como el historial de estados) y la **referenciación (referencing)** para entidades independientes.

2.1 Colección: `usuarios`

Almacena las credenciales de acceso, roles del sistema y datos de identidad del ciudadano o funcionario municipal.

- **Estructura Físico-Lógica (BSON):**

JSON

```
{
  "_id": { "$objectId": "652d8a1f3c9e8b1234567890" },
  "dni": "72849102",
  "nombres": "Juan Carlos",
  "apellidos": "Pérez Gómez",
  "correo": "juan.perez@email.com",
  "password": "hashed_password_string",
  "rol": "CIUDADANO", // Valores admitidos: CIUDADANO, ADMIN, PERSONAL
  "telefono": "+51987654321",
  "direccion": "Av. Real 123, San Agustín de Cajas",
  "fechaRegistro": { "$date": "2026-10-01T10:30:00Z" }
}
```



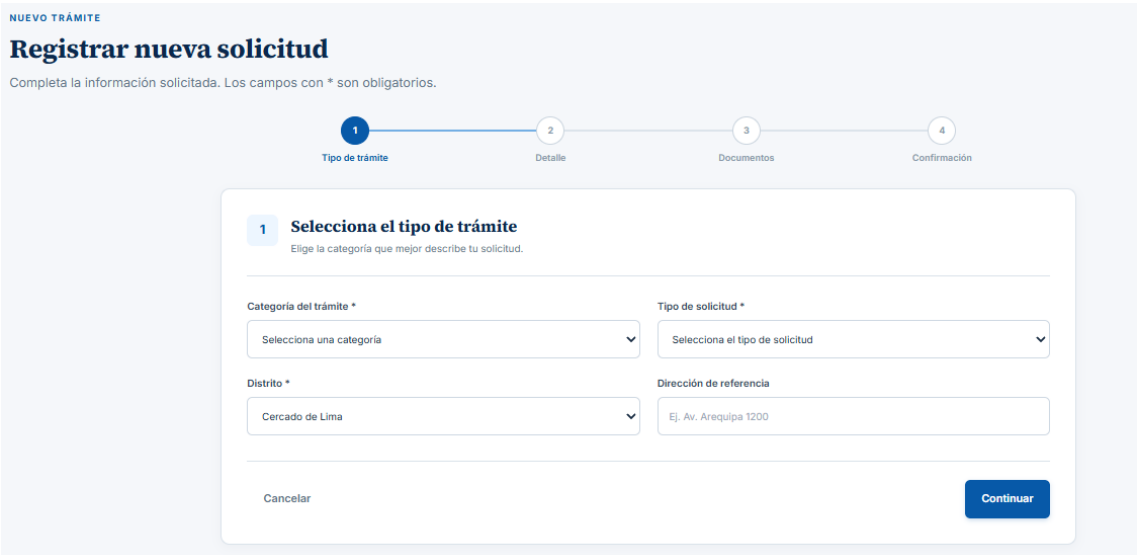
2.2 Colección: `solicitudes`

Representa el expediente digital ingresado por el ciudadano. Incluye una lista embebida de objetos `movimientos` para mantener la trazabilidad sin necesidad de realizar operaciones *JOIN* multitable.

- **Estructura Físico-Lógica (BSON):**

JSON

```
{
  "_id": { "$objectId": "652d8b4e3c9e8b1234567891" },
  "nroSolicitud": "SOL-2026-00084",
  "idUsuario": "652d8a1f3c9e8b1234567890",
  "tipoSolicitud": "LICENCIA_CONSTRUCCION",
  "asunto": "Solicitud de Licencia de Edificación Urbana",
  "descripcion": "Requerimiento de permiso para edificación de
vivienda unifamiliar...",
  "estado": "EN_PROCESO", // PENDIENTE, EN_PROCESO, OBSERVADO,
FINALIZADO
  "fechaCreacion": { "$date": "2026-10-05T09:15:00Z" },
  "documentosAdjuntos": [
    {
      "nombreArchivo": "plano_arquitectura.pdf",
      "url":
"https://sacstorage.blob.core.windows.net/docs/plano_arquitectura.pdf"
    }
  ],
  "movimientos": [
    {
      "fecha": { "$date": "2026-10-05T09:15:00Z" },
      "area": "Mesa de Partes",
      "accion": "REGISTRO_INICIAL",
      "comentario": "Solicitud ingresada por la plataforma web.",
      "usuarioResponsable": "SISTEMA"
    },
    {
      "fecha": { "$date": "2026-10-06T14:00:00Z" },
      "area": "Gerencia de Obras Privadas",
      "accion": "ASIGNACION_EVALUADOR",
      "comentario": "En revisión por el equipo técnico.",
      "usuarioResponsable": "ing_ramirez"
    }
  ]
}
```

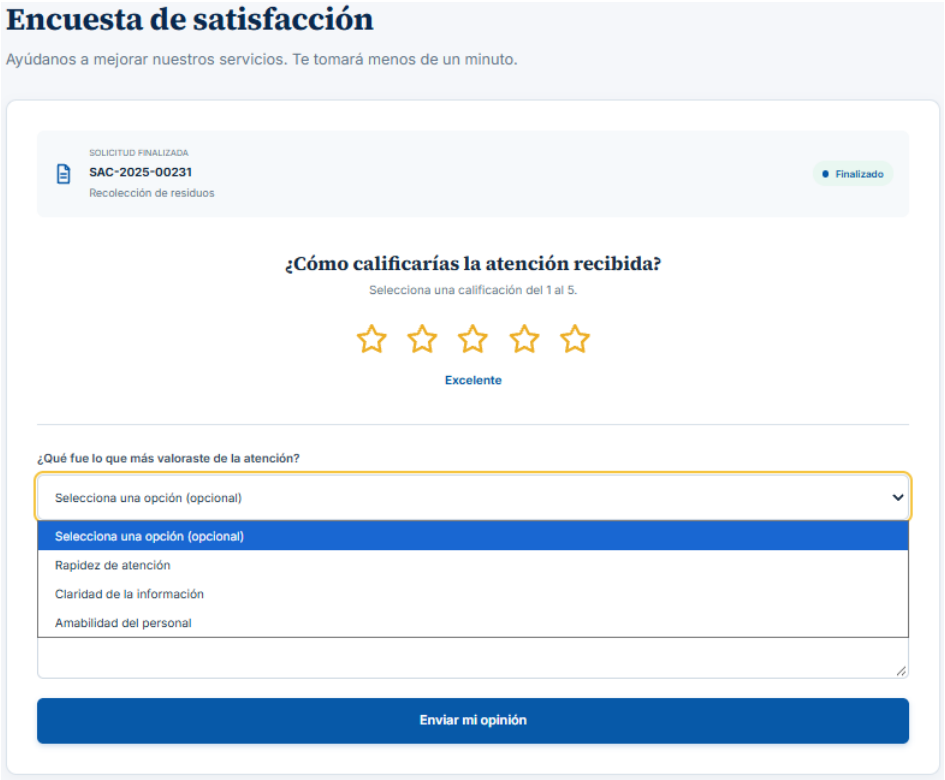


2.3 Colección: `encuestas`

Registra la evaluación de calidad del servicio prestado tras la finalización de un trámite.

- **Estructura Físico-Lógica (BSON):**

JSON
```
{
  "_id": { "$objectId": "652d8f9a3c9e8b1234567892" },
  "idSolicitud": "652d8b4e3c9e8b1234567891",
  "idUsuario": "652d8a1f3c9e8b1234567890",
  "puntuacion": 5, // Escala del 1 al 5
  "comentarios": "Excelente atención y rápida resolución del
trámite.",
  "fechaRegistro": { "$date": "2026-10-10T16:20:00Z" }
}
```



3. ESTRATEGIA DE INDEXACIÓN Y RENDIMIENTO

Para optimizar la velocidad de respuesta en los Servlets (`SolicitudServlet`, `SeguimientoServlet`, `ReporteServlet`), se aplicaron los siguientes índices físicos en MongoDB:

JavaScript
```
// Script de creación de índices (database/indices.js)

// 1. Colección usuarios: Búsqueda rápida por DNI y Correo (Unicidad
garantizada)
db.usuarios.createIndex({ "dni": 1 }, { unique: true });
db.usuarios.createIndex({ "correo": 1 }, { unique: true });
```

```
// 2. Colección solicitudes: Consultas de seguimiento por código y
listados por ciudadano/estado
db.solicitudes.createIndex({ "nroSolicitud": 1 }, { unique: true });
db.solicitudes.createIndex({ "idUsuario": 1 });
db.solicitudes.createIndex({ "estado": 1, "fechaCreacion": -1 });

// 3. Colección encuestas: Búsqueda asociativa con la solicitud
db.encuestas.createIndex({ "idSolicitud": 1 }, { unique: true });
```

4. APROVISIONAMIENTO Y CONFIGURACIÓN EN MICROSOFT AZURE

Para alojar la base de datos en la nube de nivel empresarial, se seleccionó **Azure Cosmos DB for MongoDB**.

4.1 Pasos de Despliegue en Microsoft Azure

1. **Acceso a la Consola de Azure:** Ingreso al portal de administración (`portal.azure.com`).
2. **Creación del Recurso:**
   - **Servicio:** Azure Cosmos DB.
   - **API Seleccionada:** Azure Cosmos DB for MongoDB (compatibilidad nativa con controladores de MongoDB).
   - **Grupo de Recursos (`Resource Group`):** `rg-atencion-ciudadana-prod`.
   - **Nombre de la Cuenta:** `sac-cosmos-db-server`.
   - **Ubicación:** `East US` (o la región más cercana disponible).
3. **Modo de Capacidad (Capacity Mode):** *Serverless* (ideal para el perfil de carga y entorno académico/pruebas operativas).
4. **Seguridad y Cifrado:**
   - Cifrado en reposo activo por defecto (AES-256).
   - Habilitación de protocolo seguro TLS/SSL versión 1.2 para el tráfico de red.
5. 

4.2 Cadena de Conexión Obtenida

Plaintext
```
mongodb://sac-cosmos-db-server:<PRIMARY_KEY>@sac-cosmos-db-
server.mongo.cosmos.azure.com:10255/?ssl=true&replicaSet=globaldb&retr
ywrites=false&maxIdleTimeMS=120000&appName=@sac-cosmos-db-server@
```

5. INTEGRACIÓN EN CÓDIGO FUENTE (`MongoUtil.java`)

Se modificó la clase de conexión Java `pe.gob.sac.config.MongoUtil` para leer la cadena de conexión desde las variables de entorno o la configuración de servidor en Tomcat:

Java
```
package pe.gob.sac.config;

import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoDatabase;

public class MongoUtil {
    private static MongoClient mongoClient;
    private static MongoDatabase database;

    // Configuración para Microsoft Azure Cosmos DB
    private static final String CONNECTION_STRING =
System.getenv("AZURE_COSMOS_CONN_STR") != null
        ? System.getenv("AZURE_COSMOS_CONN_STR")
        : "mongodb://localhost:27017";

    private static final String DATABASE_NAME = "sac_db";

    public static MongoDatabase getDatabase() {
        if (database == null) {
            try {
                mongoClient = MongoClients.create(CONNECTION_STRING);
                database = mongoClient.getDatabase(DATABASE_NAME);
                System.out.println("Conexión exitosa a la Base de
Datos en la Nube (Azure Cosmos DB).");
            } catch (Exception e) {
                System.err.println("Error al conectar con la base de
datos: " + e.getMessage());
            }
        }
        return database;
    }

    public static void close() {
        if (mongoClient != null) {
            mongoClient.close();
        }
    }
}
```

6. ESTRUCTURA EN EL REPOSITORIO DE CÓDIGO (GIT / GITHUB)

Para garantizar las buenas prácticas de *Infrastructure as Code* (IaC) y control de versiones, se añadió la carpeta `/database` al repositorio del proyecto:

Plaintext

```
atencion-ciudadana/
├── database/
│   ├── collections/
│   │   ├── usuarios.json          # Plantilla / Schema de la
colección Usuarios
│   │   ├── solicitudes.json       # Plantilla / Schema de la
colección Solicitudes
│   │   └── encuestas.json         # Plantilla / Schema de la
colección Encuestas
│   ├── indices.js                 # Script de automatización de
creación de índices
│   ├── seed_data.json             # Datos iniciales de prueba para el
entorno de dev
│   └── README.md                  # Documentación del diseño físico y
pasos en Azure
├── src/                           # Código fuente de la aplicación
Java EE
├── Dockerfile                     # Configuración de
contenedorización
└── pom.xml                        # Gestor de dependencias
Maven[cite: 1]
```

7. CONCLUSIONES

1. **Eficiencia en la Consulta:** El diseño físico orientado a documentos en MongoDB reduce la latencia en las vistas principales (`detalle.jsp`, `seguimiento.jsp`)[cite: 1] al almacenar los movimientos del expediente de manera embebida[cite: 1].
2. **Escalabilidad Cloud:** La migración a **Azure Cosmos DB** permite escalar dinámicamente la capacidad de cómputo y almacenamiento sin necesidad de modificar el código Java subyacente.
3. **Estandarización:** La inclusión de los scripts de la base de datos dentro del repositorio Git asegura la trazabilidad del modelo de datos junto a los cambios del software.